

Couse Code: NECE102



Dr. Swati Rajput

Assistant Professor

Department of Electronics Engineering

Indian Institute of Technology (Indian School of Mines) Dhanbad

Digital Electronics: Course Goals and Key Takeaways



Course Objective

To develop an understanding of digital electronics and its application which will be further useful to in the advanced courses of Electronics and Industrial Engineering.

Learning Outcomes

Upon successful completion of this course, students will:

- Gain information about the basic knowledge of digital electronics to design various types of digital circuits and systems.
- Learn how to analyse the performance and working of digital circuits.
- Develop their knowledge of digital electronics for industrial and robotic applications.

Digital Electronics: Key Topics at a Glance



Unit No.	Topics to be Covered	Lecture Hours	Learning Outcome
1.	Number System, Logic Gates, Boolean Algebra, NAND and NOR Implementation of Logic Functions, Determination of Boolean Functions from Digital Circuits.	08	Acquire an understanding of number system, basic logic gates and their applications to develop digital circuits and systems.
2.	Standard Representations of Logical Functions, SOP and POS form, Simplification of Logical Functions using K-map.	08	Acquire the knowledge to represent the logic functions in standard form and its minimization.
3.	Combinational Logic Circuit: Adder, Subtractor, Magnitude Comparator, Code Converters etc. MSI Circuits: Multiplexers, Demultiplexers, Decoders, Encoders etc.	08	Understand the design and applications of combinational logic circuits.
4.	Sequential Circuits: Latches and Flip-flops, Counters, Shift Registers.	10	Acquire the ability to design and perform the analysis of different sequential circuits.
5.	Analysis of Sequential circuits: State tables and state diagrams, Timing Circuits	04	Acquire an understanding on the analysis of sequential circuits.
6.	Introduction to logic families, semiconductor memories, ROM, PAL, PLA and Microprocessor	04	Develop knowledge about the various techniques for IC designing for digital circuits.

Textbook:

1. Floyd T.L., Digital Fundamentals, 10/2, Pearson Education, 2011.

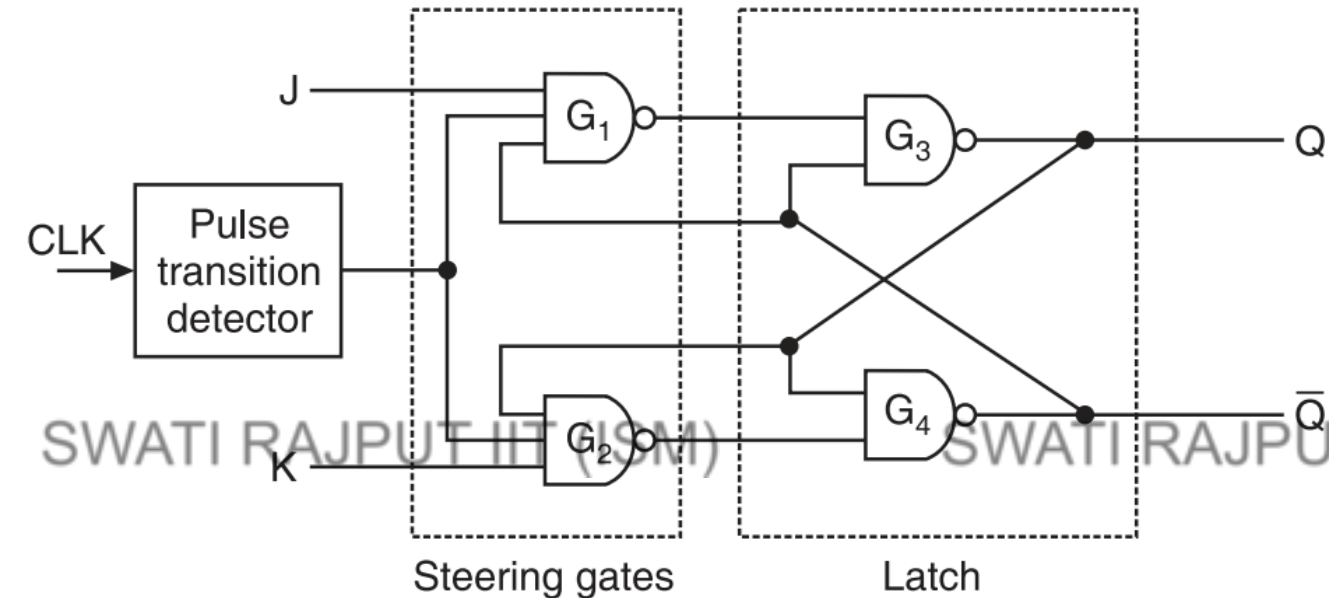
Reference Books:

1. Digital Logic and Computer Design by M. Morris Mano, Prentice –Hall, 2015.
2. Fundamental of Digital Electronics by A. Anand Kumar, 4/e Prentice Hall, 2016.

Race Around Condition in JK Flip Flop



What if Flip Flops were Level Triggered????



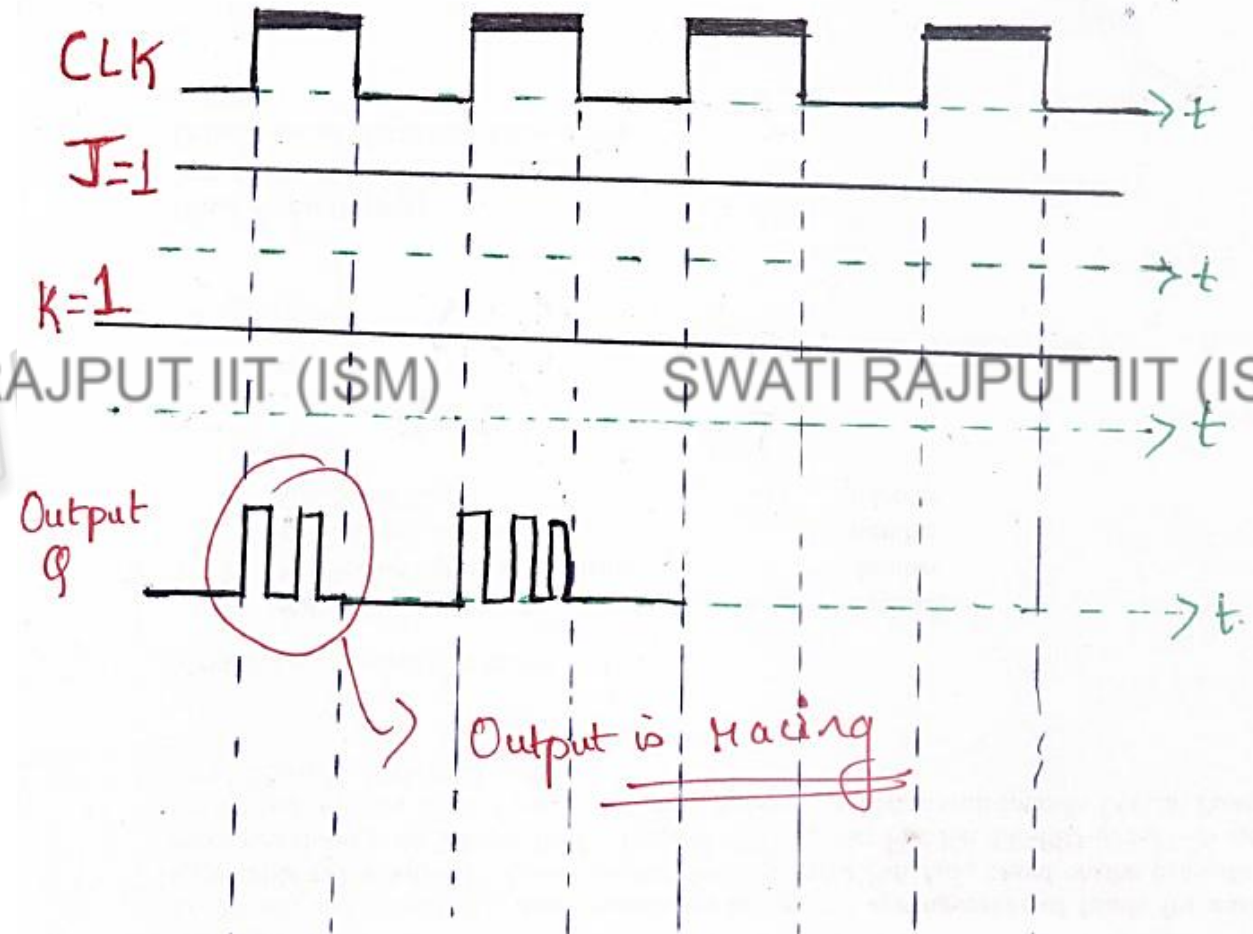
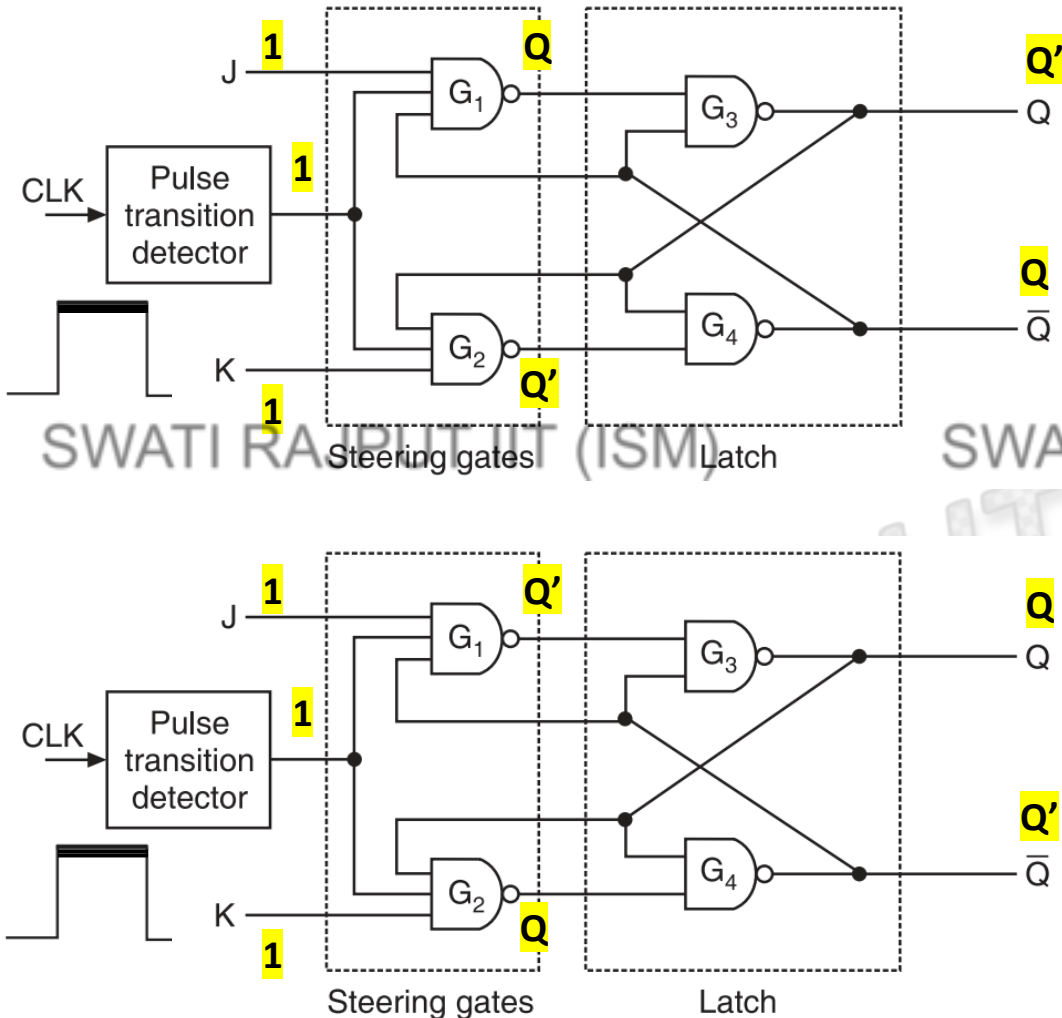
Truth Table of JK Flip Flop

C	J	K	Q_n	Q_{n+1}
$\uparrow$	0	0	0	0
$\uparrow$	0	0	1	1
$\uparrow$	0	1	0	0
$\uparrow$	0	1	1	0
$\uparrow$	1	0	0	1
$\uparrow$	1	0	1	1
$\uparrow$	1	1	0	1
$\uparrow$	1	1	1	0
0	x	x	0	0
0	x	x	1	1

Race Around Condition in JK Flip Flop



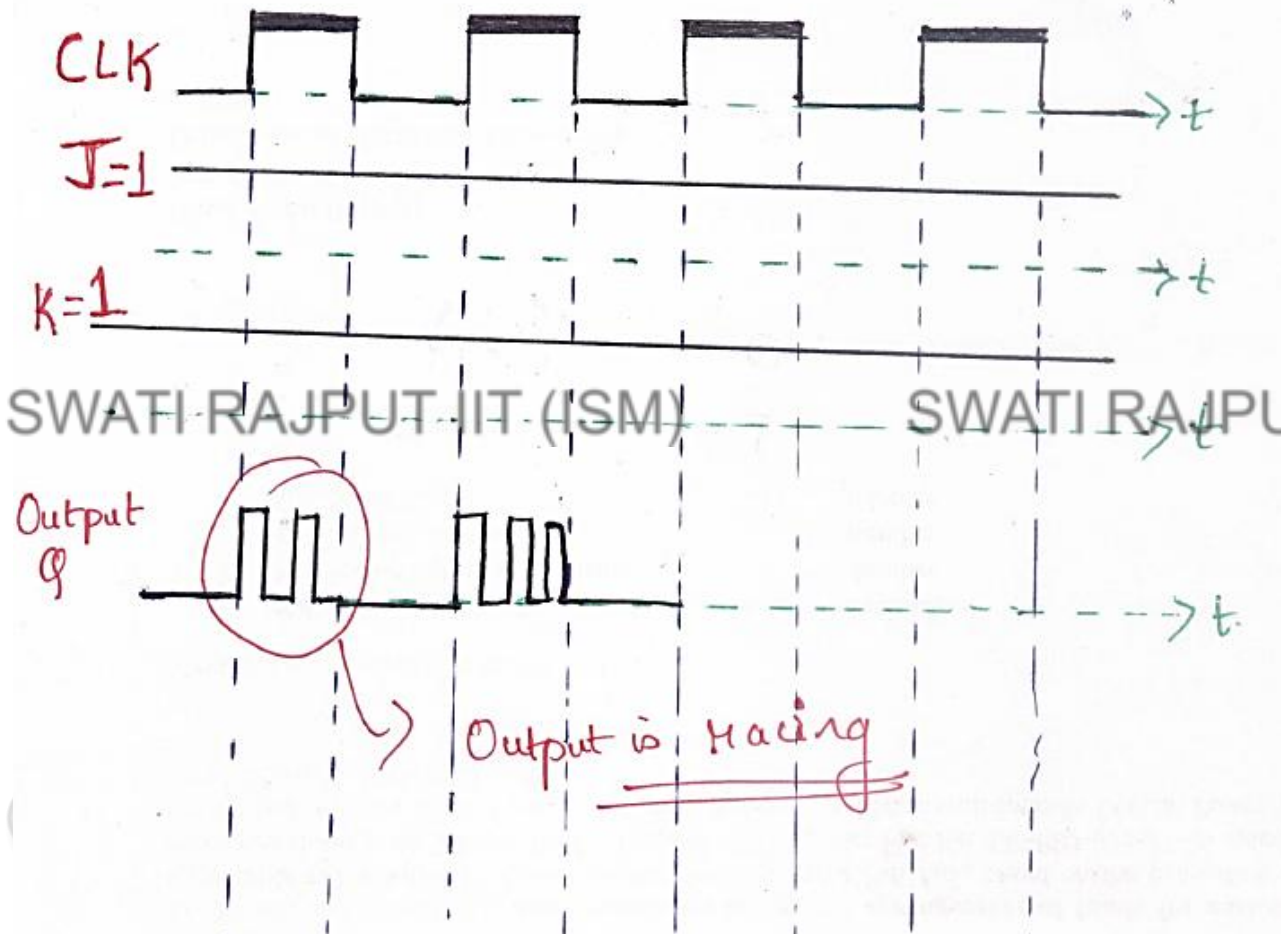
What if Flip Flops were Level Triggered????



Race Around Condition in JK Flip Flop

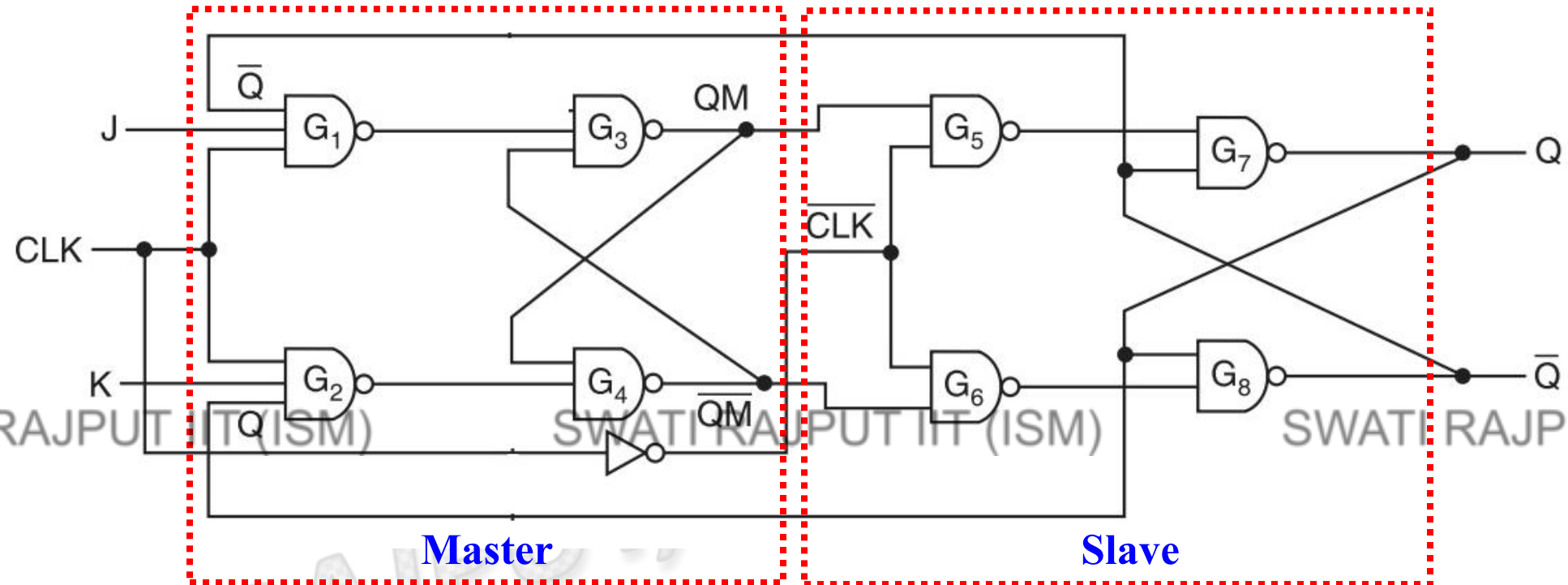


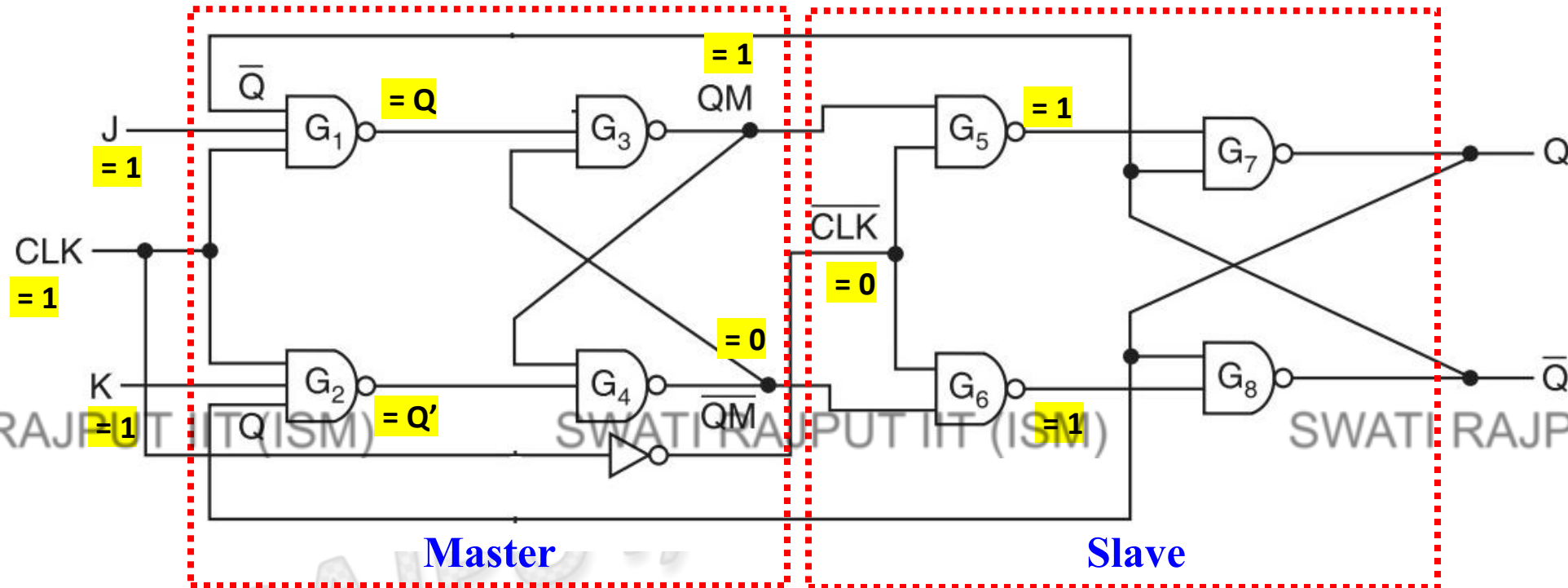
What if Flip Flops were Level Triggered????



Let us consider the inputs $J = 1$ and $K = 1$, and the output $Q = 0$. After the propagation delay (let t) of the flip-flops, the output of the JK flip-flops changes from 0 to 1. As we know, the output of the JK flip-flops is connected to its inputs. Hence, the output also acts as input, and thus after the next delay (t), the output will change from 1 to 0.

This process will continue till the end of the applied clock signal. Thus, the output of the JK flip-flops is uncertain. This condition of JK flip-flops is called the **race-around condition**.





Case 1: Clk of Master = 1; J = 1; K = 1; Clock of Slave = 0

Output of G_1 is Q

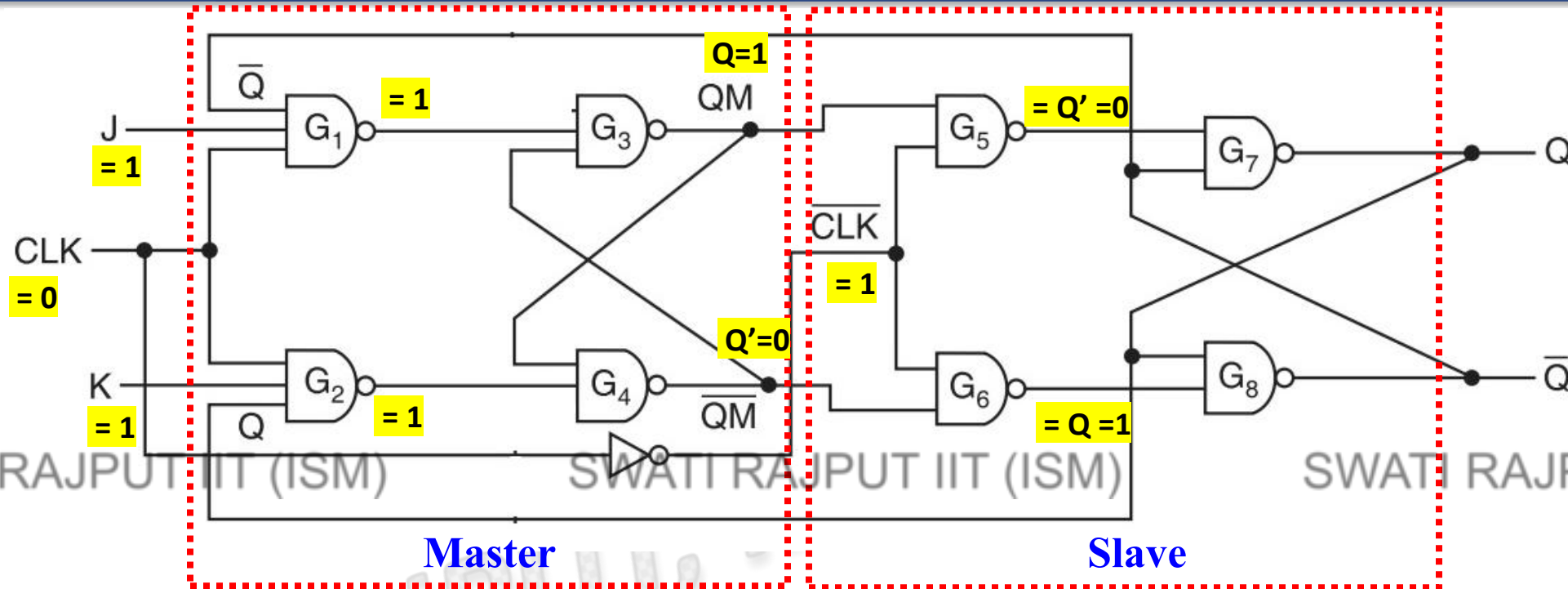
Output of G_2 is $\overline{Q}$

If $Q = 0$; Q' is 1; Output of G_3 is 1 $\rightarrow$ Toggle

Output of G_4 is 0

Slave will be in Hold State; feedback is not fluctuating again and again thus race around condition will not come.

Master Slave JK Flip Flop



Case 2: Clk of Master =0; J = 1; K = 1; Clock of Slave = 1

Output of G_1 is 1

Output of G_2 is 1

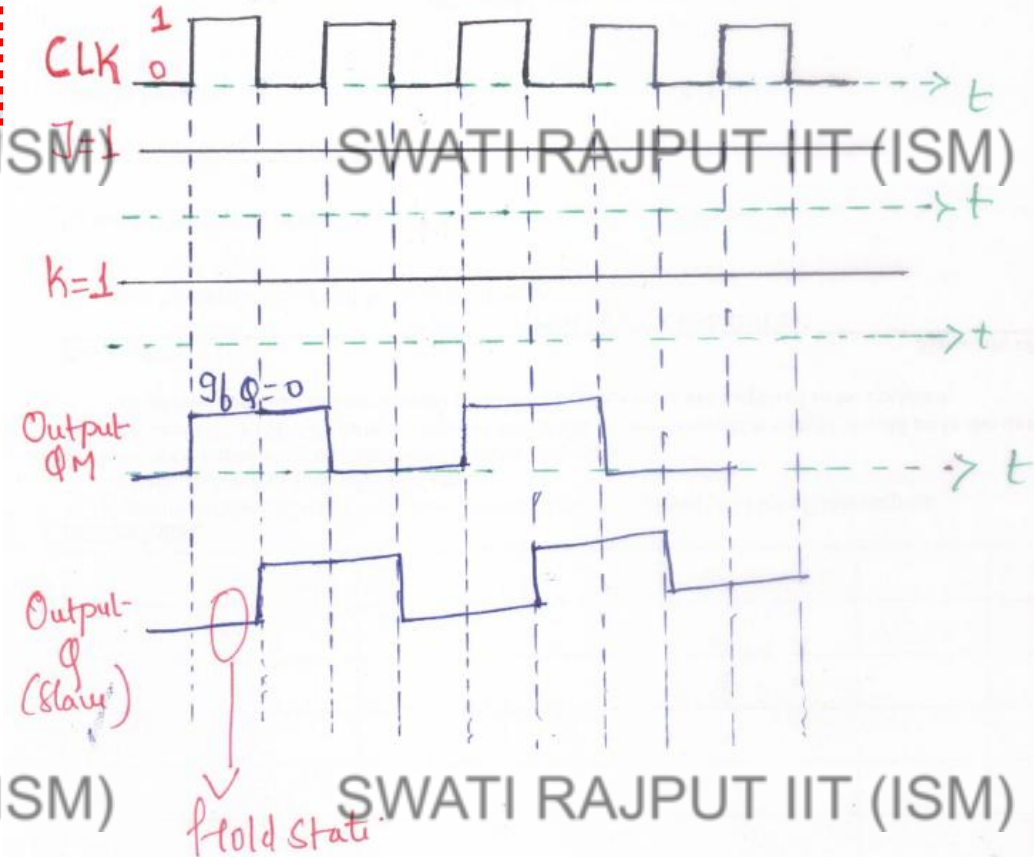
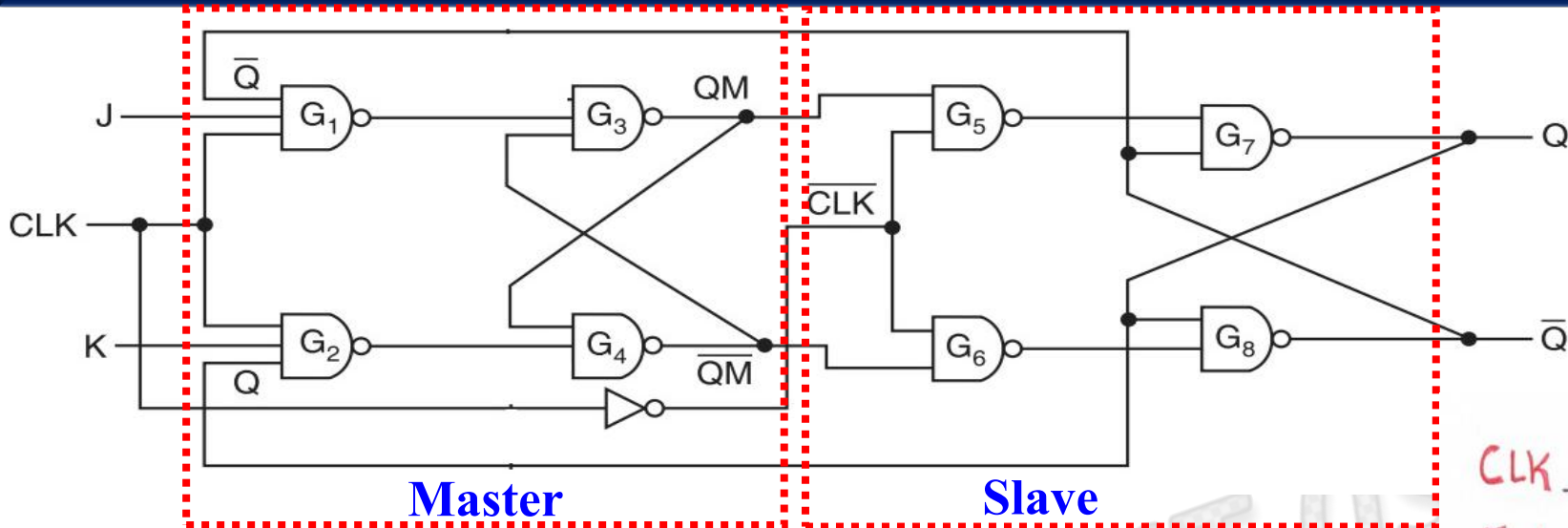
Master will be in Hold State

Output of G_5 is Q'

Output of G_6 is Q'

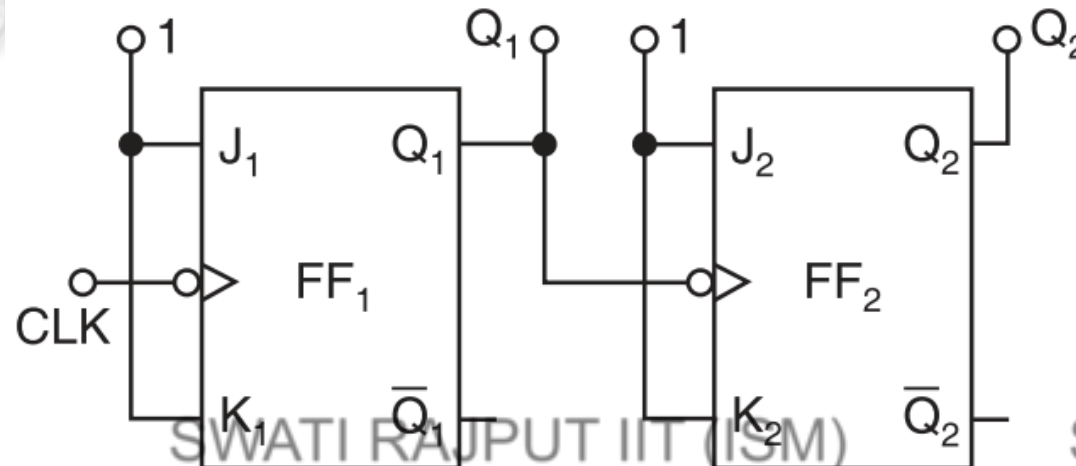
Output of G_7 is 1 → Toggle

Output of G_8 is 0



Counter

- ❖ A counter is a device that stores (and sometimes displays) the number of times a particular event or process has occurred, often in relationship to a clock.
- ❖ A counter circuit is usually constructed of a number of flip flops connected in cascade.
- ❖ A digital counter is a set of flip-flops (FFs) whose states change in response to pulses applied at the input to the counter.
- ❖ The FFs are interconnected such that their combined state at any time is the binary equivalent of the total number of pulses that have occurred up to that time. Thus, as its name implies, a counter is used to count pulses.
- ❖ A counter can also be used as a frequency divider to obtain waveforms with frequencies that are specific fractions of the clock frequency.



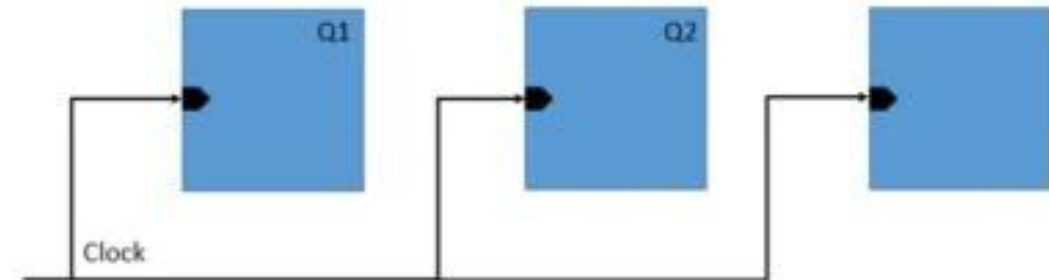
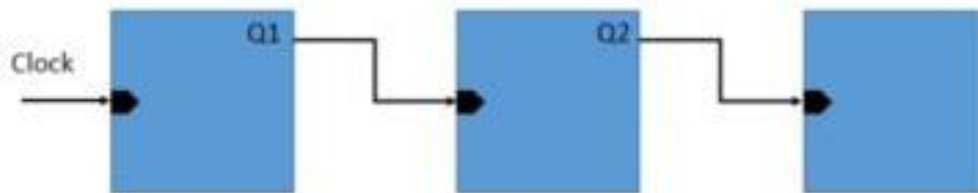
Counter

Asynchronous counters

1. In this type of counter FFs are connected in such a way that the output of first FF drives the clock for the second FF, the output of the second the clock of the third and so on.
2. All the FFs are not clocked simultaneously.
3. Design and implementation is very simple even for more number of states.
4. Main drawback of these counters is their low speed as the clock is propagated through a number of FFs before it reaches the last FF.

Synchronous counters

1. In this type of counter there is no connection between the output of first FF and clock input of next FF and so on.
2. All the FFs are clocked simultaneously.
3. Design and implementation becomes tedious and complex as the number of states increases.
4. Since clock is applied to all the FFs simultaneously the total propagation delay is equal to the propagation delay of only one FF. Hence they are faster.





Counter

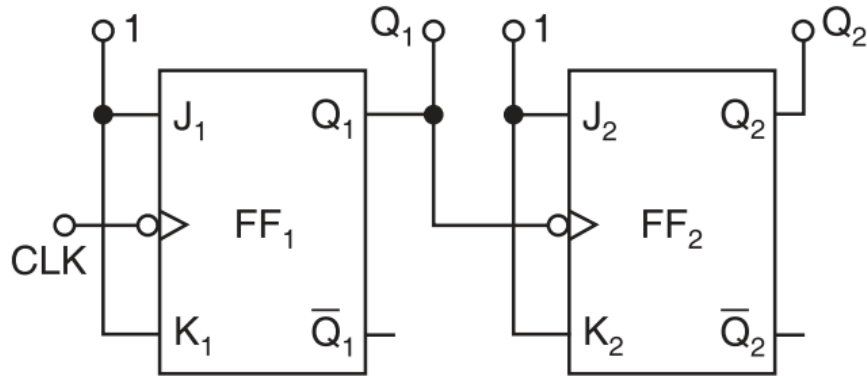
- ❖ A counter may be an *up-counter* or a *down-counter*.
- ❖ An up-counter is a counter which counts in the upward direction, i.e. $0, 1, 2, 3, \dots, N$.
- ❖ A down-counter is a counter which counts in the downward direction, i.e. $N, N-1, N-2, N-3, \dots, 1, 0$.
- ❖ Each of the counts of the counter is called the *state of the counter*.
- ❖ The number of states through which the counter passes before returning to the starting state is called the *modulus of the counter*. Hence, the modulus of a counter is equal to the total number of distinct states (counts) including zero that a counter can store.
- ❖ In other words, the number of input pulses that causes the counter to reset to its initial count is called the modulus of the counter.
- ❖ Since a 2-bit counter has 4 states, it is called a mod-4 counter.
- ❖ It divides the input clock signal frequency by 4, therefore, it is also called a divide-by-4 counter. It requires two FFs.
- ❖ Similarly, a 3-bit counter uses 3 FFs and has $2^3 = 8$ states. It divides the input clock frequency by 2^3 , i.e. 8.
- ❖ In general, an n -bit counter will have n FFs and 2^n states, and divides the input frequency by 2^n . Hence, it is a divide-by- 2^n counter.



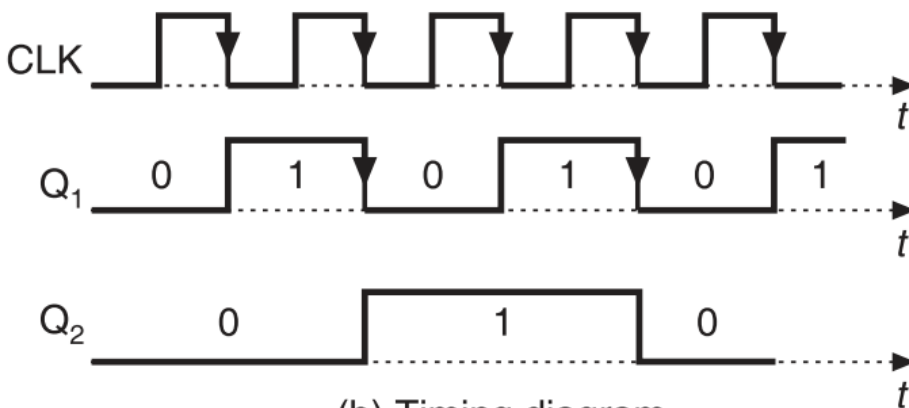
- ❖ A counter may have a **shortened modulus**. This type of counter does not utilize all the possible states.
- ❖ **Some of the states are unutilized, i.e. invalid.**
- ❖ The number of FFs required to construct a mod- N counter equals the smallest n for which $N \leq 2^n$.
- ❖ A mod- N counter divides the input frequency by N , hence, it is called a divide-by- N counter.



Two-Bit Ripple Up-Counter using Negative Edge Triggered Flip Flops



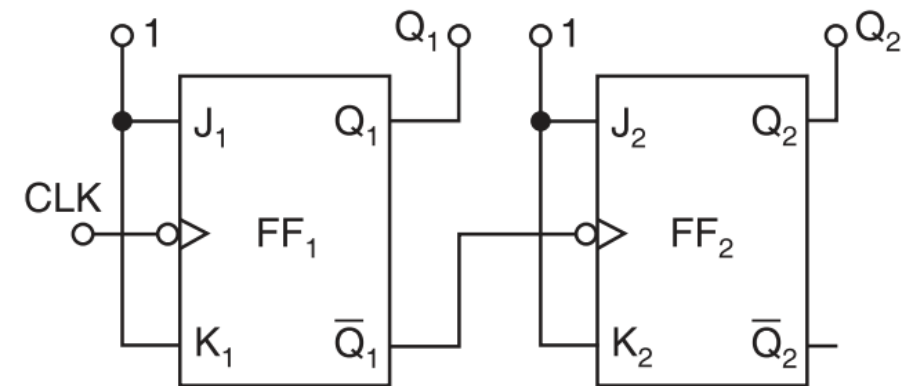
(a) Logic diagram



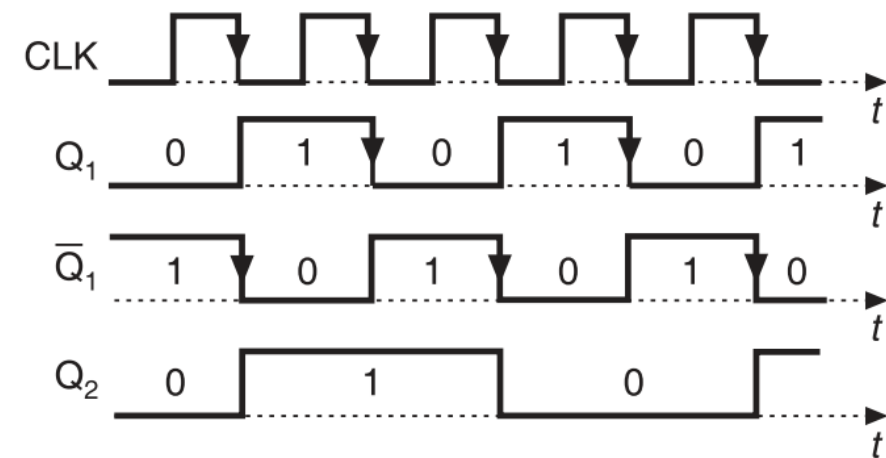
(b) Timing diagram

- ❖ The 2-bit up-counter counts in the order 0, 1, 2, 3, 0, 1, ..., i.e. 00, 01, 10, 11, 00, 01, ..., etc.
- ❖ The counter is initially reset to 00.
- ❖ When the first clock pulse is applied, FF₁ toggles at the negative-going edge of this pulse, therefore, Q₁ goes from LOW to HIGH.
- ❖ This becomes a positive going signal at the clock input of FF₂. So, FF₂ is not affected, and hence, the state of the counter after one clock pulse is Q₁ = 1 and Q₂ = 0, i.e. 01.
- ❖ At the negative-going edge of the second clock pulse, FF₁ toggles. So, Q₁ changes from HIGH to LOW and this negative-going signal applied to CLK of FF₂ activates FF₂, and hence, Q₂ goes from LOW to HIGH. Therefore, Q₁ = 0 and Q₂ = 1, i.e. 10 is the state of the counter after the second clock pulse.
- ❖ At the negative-going edge of the third clock pulse, FF₁ toggles. So Q₁ changes from a 0 to a 1. This becomes a positive-going signal to FF₂, hence, FF₂ is not affected. Therefore, Q₂ = 1 and Q₁ = 1, i.e. 11 is the state of the counter after the third clock pulse.
- ❖ At the negative-going edge of the fourth clock pulse, FF₁ toggles. So, Q₁ goes from a 1 to a 0. This negative-going signal at Q₁ toggles FF₂, hence, Q₂ also changes from a 1 to a 0. Therefore, Q₂ = 0 and Q₁ = 0, i.e. 00 is the state of the counter after the fourth clock pulse.
- ❖ For subsequent clock pulses, the counter goes through the same sequence of states. So, it acts as a mod-4 counter with Q₁ as the LSB and Q₂ as the MSB. The counting sequence is thus 00, 01, 10, 11, 00, 01, ..., etc.

Two-Bit Ripple Down Counter using Negative Edge Triggered Flip Flops



(a) Logic diagram

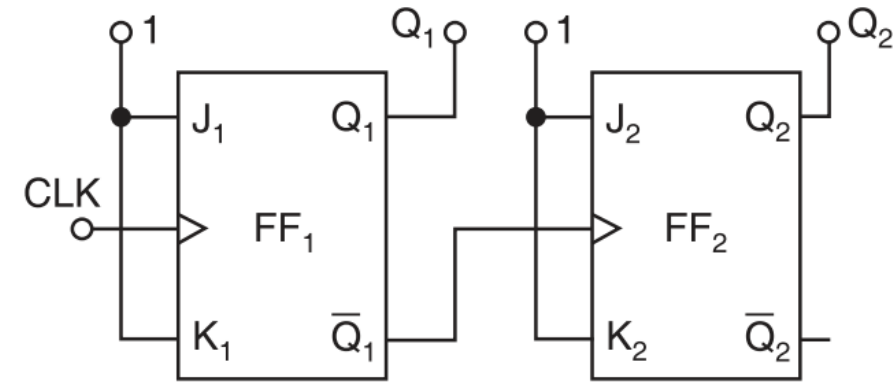


(b) Timing diagram

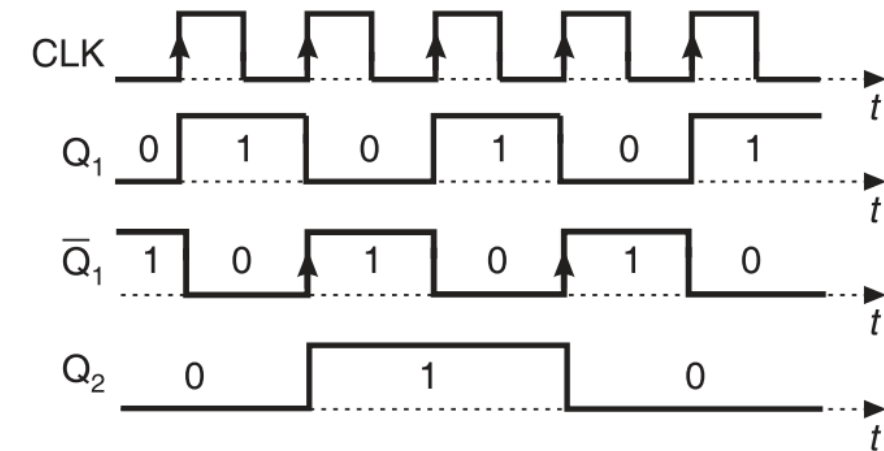
- ❖ A 2-bit down-counter counts in the order 0, 3, 2, 1, 0, 3, ..., i.e. 00, 11, 10, 01, 00, 11, ..., etc.
- ❖ For down counting, $\overline{Q_1}$ of FF₁ is connected to the clock of FF₂.
- ❖ Let initially all the FFs be reset, i.e. let the count be 00.
- ❖ At the negative-going edge of the first clock pulse, FF₁ toggles, so, Q_1 goes from a 0 to a 1 and $\overline{Q_1}$ goes from a 1 to a 0. This negative-going signal at $\overline{Q_1}$ applied to the clock input of FF₂, toggles FF₂ and, therefore, Q_2 goes from a 0 to a 1. So, after one clock pulse $Q_2 = 1$ and $Q_1 = 1$, i.e. the state of the counter is 11.
- ❖ At the negative-going edge of the second clock pulse, Q_1 changes from a 1 to a 0 and $\overline{Q_1}$ from a 0 to a 1. This positive-going signal at $\overline{Q_1}$ does not affect FF₂ and, therefore, Q_2 remains at a 1. Hence, the state of the counter after the second clock pulse is 10.
- ❖ At the negative-going edge of the third clock pulse, FF₁ toggles. So, Q_1 goes from a 0 to 1 and $\overline{Q_1}$ from a 1 to a 0. This negative-going signal at $\overline{Q_1}$ toggles FF₂ and, so, Q_2 changes from a 1 to a 0. Hence, the state of the counter after the third clock pulse is 01.
- ❖ At the negative going edge of the fourth clock pulse, FF₁ toggles. So, Q_1 goes from a 1 to a 0 and $\overline{Q_1}$ from a 0 to 1. This positive-going signal at $\overline{Q_1}$ does not affect FF₂. So, Q_2 remains at a 0. Hence, the state of the counter after the fourth clock pulse is 00.
- ❖ For subsequent clock pulses the counter goes through the same sequence of states, i.e. the counter counts in the order 00, 11, 10, 01, 00, and 11 ...



Two-Bit Ripple Up-Counter using Positive Edge Triggered Flip Flops



(a) Logic diagram

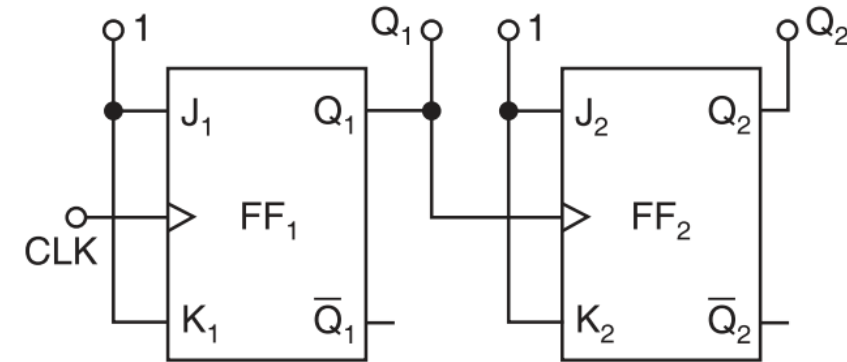


(b) Timing diagram

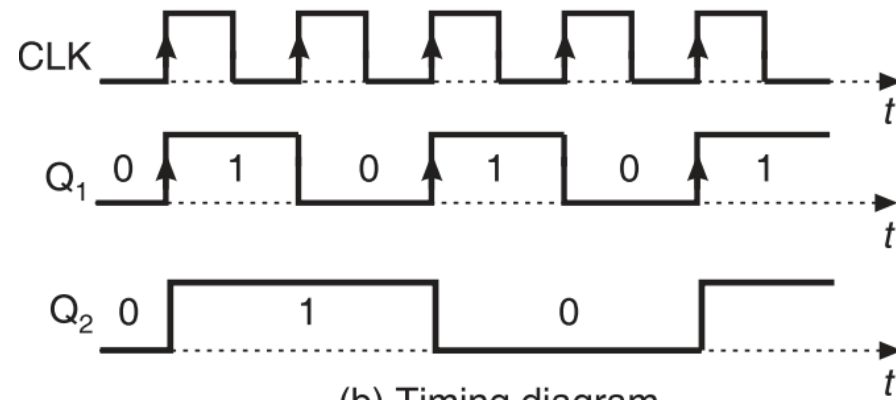
- ❖ The $\overline{Q_1}$ output of the first FF₁ is connected to the clock of FF₂.
- ❖ The external clock signal is applied to the first flip-flop FF₁.
- ❖ The FF₁ toggles at the positive-going edge of each clock pulse and FF₂ toggles whenever $\overline{Q_1}$ changes from a 0 to a 1.
- ❖ State transitions occur at the positive-going edges of the clock pulses.
- ❖ The counting sequence is 00, 01, 10, 11, 00, 01, ..., etc

Asynchronous Counter

Two-Bit Ripple Down Counter using Positive Edge Triggered Flip Flops



(a) Logic diagram

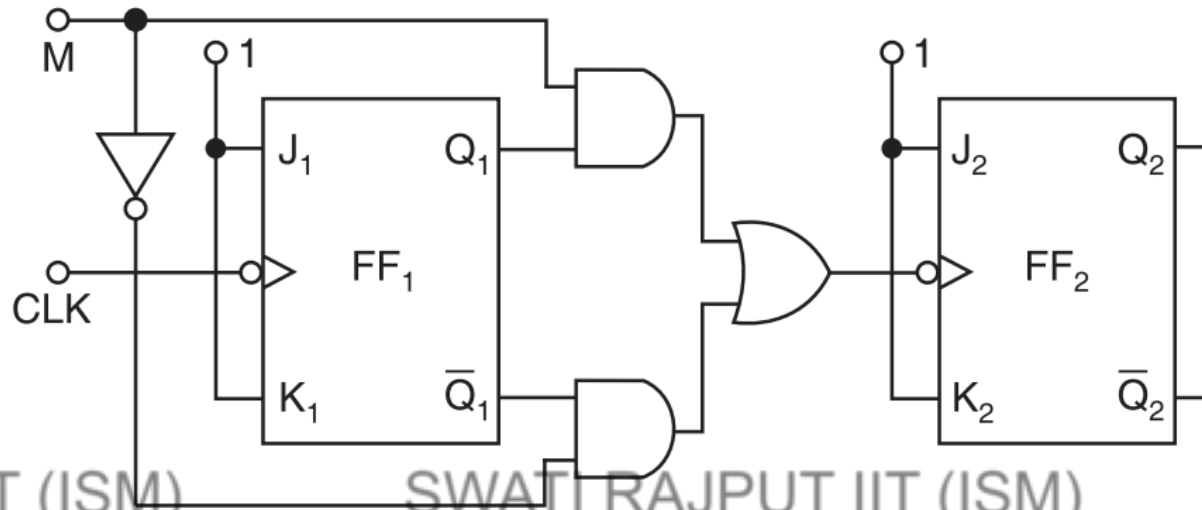


(b) Timing diagram

- ❖ The Q_1 output of the first FF is connected to the clock of FF_2 .
- ❖ The external clock signal is applied to FF_1 .
- ❖ The FF_1 toggles at the positive-going edge of each clock pulse.
- ❖ The FF_2 toggles whenever Q_1 changes from a 0 to a 1.
- ❖ The counting sequence is 00, 11, 10, 01, 00, 11..., etc.

Asynchronous Counter

Two-Bit Ripple Up-Down Counter using Negative Edge Triggered Flip Flops



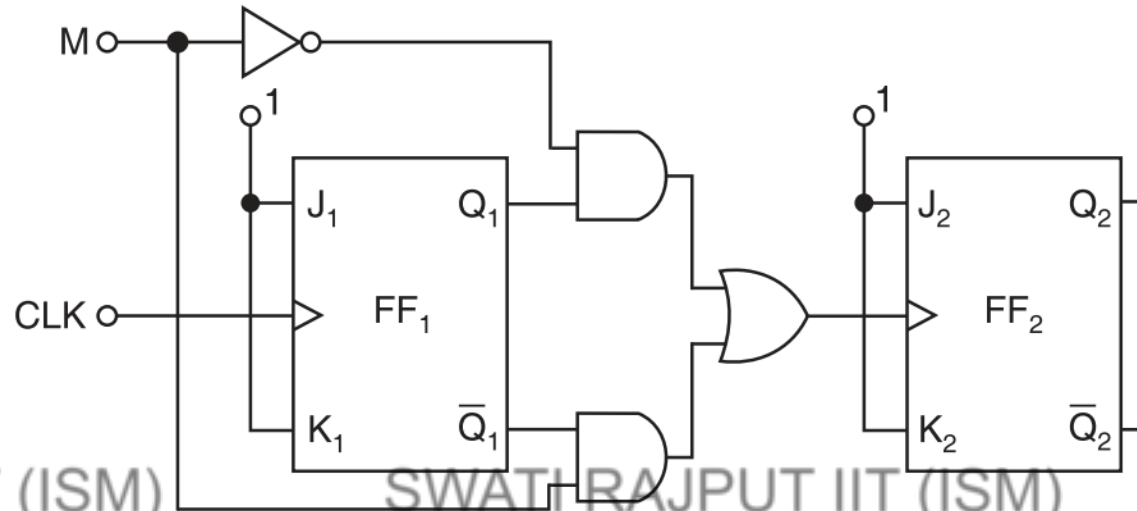
Asynchronous 2-bit up-down counter using negative edge-triggered flip-flops.

$$\text{Clock signal to FF}_2 = (Q_1 \cdot \text{Up}) + (\bar{Q}_1 \cdot \text{Down}) = Q_1 M + \bar{Q}_1 \bar{M}$$

- ❖ An up-down counter is a counter which can count both in upward and downward directions.
- ❖ An up-down counter is also called a forward/backward counter or a bidirectional counter.
- ❖ So, a control signal or a mode signal M is required to choose the direction of count.
- ❖ When M = 1 for up counting, Q₁ is transmitted to clock of FF₂ and when M = 0 for down counting, $\bar{Q}_1$ is transmitted to clock of FF₂.
- ❖ This is achieved by using two AND gates and one OR gate.
- ❖ The external clock signal is applied to FF₁.

Asynchronous Counter

Two-Bit Ripple Up-Down Counter using Positive Edge Triggered Flip Flops



Logic diagram of a two-bit ripple up/down counter using positive edge-triggered flip-flops.

$$\text{Clock signal to FF}_2 = (\bar{Q}_1 \cdot \text{Up}) + (Q_1 \cdot \text{Down}) = \bar{Q}_1 M + Q_1 \bar{M}$$

- ❖ A 2-bit ripple up/down counter using positive edge-triggered J-K FFs.
- ❖ When $M = 1$ for up counting, $\bar{Q}_1$ is transmitted to the clock of FF_2 and when $M = 0$ for down counting, Q_1 is transmitted to the clock of FF_2 . This is achieved by using two AND gates and one OR gate.



Preset and Clear Inputs in Flip Flops

1.Preset:

- A "preset" input is used to set the flip-flop to a predefined state, typically 1. When the preset is activated (often by applying a logic high signal to the preset pin), the flip-flop is forced into the "set" condition, regardless of the other inputs.

2.Clear:

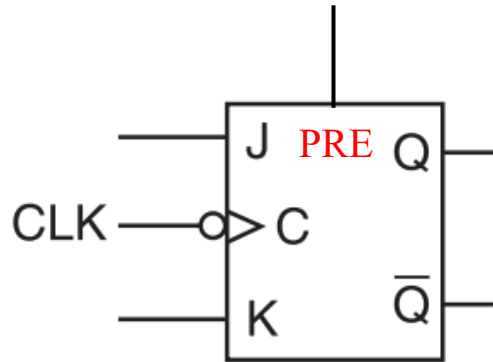
- The "clear" input is used to reset the flip-flop to a predefined state, typically 0. When the clear input is activated (by applying a logic high signal to the clear pin), the flip-flop is forced into the "reset" condition, regardless of the other inputs.

In short:

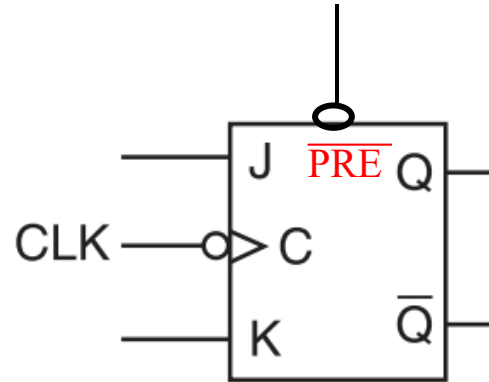
- **Preset** sets the output to 1 ($Q_n = 1$)
- **Clear** resets the output to 0 ($Q_n = 0$)



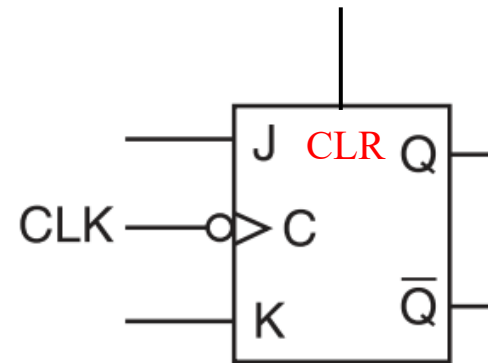
Preset and Clear Inputs in Flip Flops



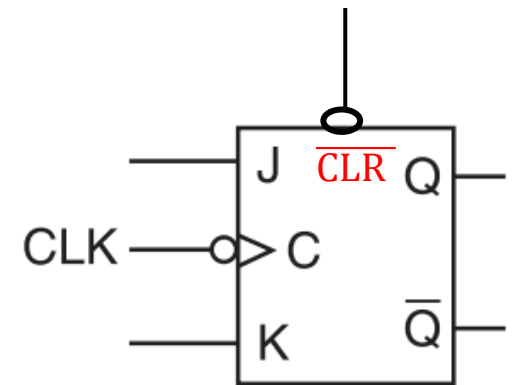
Active High Set



Active Low Set



Active High Reset



Active Low Reset

Mod-6 Asynchronous Counter using Flip Flops



- ❖ A mod-6 counter has six stable states 000, 001, 010, 011, 100, and 101.
- ❖ When the sixth clock pulse is applied, the counter temporarily goes to 110 state, but immediately resets to 000 because of the feedback provided.
- ❖ It is a 'divide-by-6 counter', in the sense that it divides the input clock frequency by 6.
- ❖ It requires three FFs, because the smallest value of n satisfying the condition $N \leq 2^n$ is $n = 3$; three FFs can have eight possible states, out of which only six are utilized and the remaining two states 110 and 111, are invalid.
- ❖ If initially the counter is in 000 state, then after the first clock pulse it goes to 001, after the second clock pulse, it goes to 010, and so on. After the sixth clock pulse, it goes to 000.

After pulses	State			R
	Q_3	Q_2	Q_1	
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
	↓	↓	↓	
	0	0	0	0
7	0	0	1	0

Mod-6 Asynchronous Counter using Flip Flops



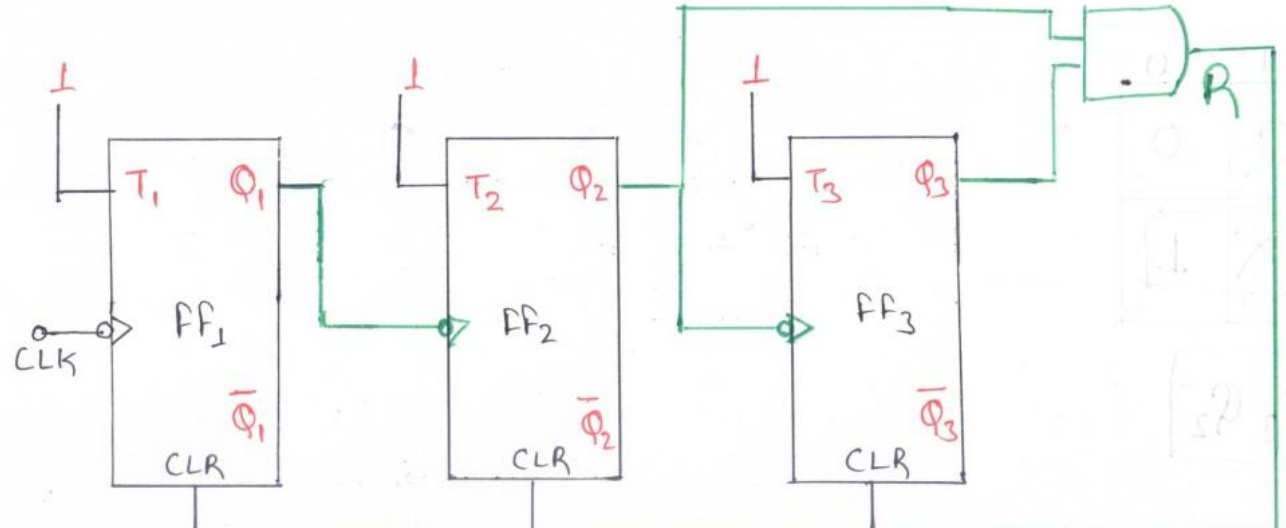
After pulses	State			R
	Q ₃	Q ₂	Q ₁	
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
↓ ↓ ↓				
7	0	0	1	0

Q ₃ \ Q ₂ Q ₁	00 01 11 10			
	0	1	3	2
0	0	1	3	2
1	4	5	7 (X)	6 (1)

$$R = Q_3 Q_2$$

therefore,

$$R = Q_3 Q_2 \bar{Q}_1 + Q_3 Q_2 Q_1 = Q_3 Q_2$$



R = 0 for 000 to 101, R = 1 for 110, and R = X for 111

